be "3" in the controller. This, by the way, is a welcome change in recent versions of Rails; older versions did not do this conversion, which led to heads occasionally pounding against walls.

The third and fourth arguments to these controller methods are optional and rarely used. The third argument sets key/value pairs for the session object, which is useful in testing multistep processes that use the session for continuity. The fourth argument represents the Rails flash object, which is useful … well, never, but if for some reason the incoming flash is important for your logic, there it is.

You may occasionally want to do something fancier to the simulated request. In a controller test you have access to the request object as @request, and access to the controller object as @controller. (As you'll see in *Evaluating Controller Results, on page 228*, you also have the @response object.) You can get at the HTTP headers using the hash @request.headers.

There is one more controller action method: xml_http_request (also aliased to xhr). This simulates a classic Ajax call to the controller and has a slightly different signature:

```
it "makes an ajax call" do
  xhr :post, :create, :task => {:id => "3"}
end
```

The method name is xhr, the first argument is the HTTP verb associated with the xhr call, and the remaining arguments are the arguments to all the other controller-calling methods in the same order: action, params, session, and flash. The xhr call sets the appropriate headers such that the Rails controller will appropriately be able to consider the request an Ajax request (meaning .js format blocks will be triggered), then simulates the call based on its arguments.

Evaluating Controller Results

Rails 4 controller tests have a few other features that were deprecated in Rails 5. In Rails 4 you could use the render_template matcher to assert which template rendered a method. I didn't use this much.

Rails 4 also exposed an assigns hash that had all the instance values that the controller passes to the view. This is deprecated now, but it was at one time a very common way to test Rails controllers. RSpec controller tests do not render the view by default, and you should use request or system specs if you would like that behavior.